PROJECT REPORT / 2026

Pastebox

A compact, full-stack text sharing application built with React, Express, and PostgreSQL.

Purpose
Create, store, and share text through short links, with optional time and view-count expiry. New pastes are private unless the creator explicitly makes them discoverable.

8 letters	500 KB	17	1
share ID length	maximum paste text	performance checklist items addressed	production web service

Project at a glance

Area	Implementation
Frontend	React 18, React Router, TanStack Query, Vite
Backend	Node.js and Express REST API
Database	Managed PostgreSQL on Neon; pg.Pool connection pool
Hosting	Render Node web service; static client served by Express
Repository	github.com/Dinesh-Karthik7736/Pastebin-Lite
Reported live URL	https://pastebox-w5qm.onrender.com

This report records repository implementation and local verification as of the date above. It does not claim a production penetration test, load test, or current external uptime check.

1. Product behavior

Pastebox provides a focused flow for creating a text paste, choosing optional expiry controls, and sharing a compact link. A paste can have a title, up to 500 KB of content, an optional expiration (1 hour, 1 day, 7 days, or 30 days), and an optional maximum number of views. A paste is private by default; an explicit Explore option makes it discoverable.

- Share IDs are eight URL-safe letters generated using cryptographic randomness and unbiased rejection sampling.
- Opening a valid link records a view. When the final allowed view is served, the paste row is deleted within the same database transaction; later requests receive not found.
- Expired and exhausted pastes cannot be read. Explore excludes private, expired, and view-exhausted pastes.
- The Explore page supports title-prefix search and bounded pagination. The interface provides copy actions, validation, error states, and loading skeletons.

2. Project structure

Path	Responsibility
client/src/main.jsx	React entry, routes, query provider, lazy route loading
client/src/pages/	Create, view, and Explore experiences; UI tests
client/src/components/	Shared header and loading skeleton
client/src/lib/api.js	Browser API request helpers
client/vite.config.js	Development proxy, production chunking and minification
server/src/app.js	Express middleware and REST endpoints
server/src/db.js	PostgreSQL pool and schema initialization
server/src/schema.sql	Table definition and query indexes
server/src/cache.js	Optional Redis cache and bounded memory fallback
server/src/startup.js	Retry helper for transient database startup failures
server/test/	API/database integration and startup retry tests
render.yaml	Render web service build, start, health-check and environment config
docker-compose.yml	Local PostgreSQL and Redis services

3. Architecture and data flow

Request flow

Create: React form validates input and sends JSON to `POST /api/pastes`. Express validates with Zod, rate limits, inserts a row through a parameterized PostgreSQL query, invalidates Explore caches, and returns the short ID and relative share path.

Read: `GET /api/pastes/:id` begins a transaction, locks the row with `FOR UPDATE`, checks expiry and view limits, increments the view count or deletes the record after its final permitted view, commits, and returns content with `Cache-Control: no-store`.

Explore: `GET /api/pastes?page=&limit=&q=` runs a single bounded SQL query for public previews. Results are cached briefly at the server and may be shared across instances through Redis. Public cache headers allow short browser/CDN caching.

Data and connection efficiency

- One `pastes` table stores the text, timestamps, expiry, view limit, view count, and discoverability.
- Indexes: primary key on share ID; partial ordered index for discoverable feed; partial lower(title) pattern index for prefix search; partial index on non-null expiry timestamps.
- A single PostgreSQL pool per Node process uses a default maximum of five connections (configurable 1–20), short connection and idle timeouts, a 30-minute maximum connection lifetime, statement timeout, lock timeout, and idle-transaction timeout. TLS certificate verification is enabled when `DATABASE_SSL=true`.
- Explore reads a page and previews in one query: there is no per-row follow-up query (no N+1 query path).
- View reads deliberately avoid cache because every successful read must be counted; database row locking protects the maximum view count under concurrent requests.

Caching boundaries

TanStack Query provides browser-side response caching for list data. Express maintains a short-lived application cache using Redis when REDIS_URL is configured, otherwise a size-bounded in-memory TTL cache. Creating or exhausting a discoverable paste invalidates Explore cache entries. Paste bodies are not cached. The in-memory fallback is per process; multi-instance deployments should use a shared cache.

4. Security and reliability controls

Control	What is implemented
Input validation	Zod schemas constrain title, content, expiry, views, pagination and search; request JSON body is bounded.
SQL safety	Database values use parameterized queries.
HTTP hardening	Helmet security headers, disabled Express identifying header, gzip/Brotli-capable compression middleware.
Rate limiting	Paste creation: 20 requests per client IP per 15 minutes. Paste reads: 120 per IP per minute. Includes Retry-After on rejection.
Link/host safety	Share URLs are relative paths; an untrusted Host header is not copied into a link. CORS is disabled unless one trusted origin is configured.
View/expiry handling	Transactional lock and expiry checks; sensitive paste responses use no-store.
Startup resilience	Schema initialization retries transient startup/database failures with capped backoff; graceful shutdown closes server and pool.
Secrets	Database URL is configured as a deployment environment secret. No secret values are included in this report.

Scope: These controls and automated cases are useful baseline protections. The project has not been subjected to a formal penetration test, dependency audit, fuzz test, or production traffic/load test as part of the verification recorded here.

5. Performance checklist

Requested item	Implementation / status
API response caching	TanStack Query for list data; short public Explore cache headers
Load balancer	Stateless Express service can run behind Render/platform HTTP load balancing; no multi-instance load test recorded
Database indexes	Primary key plus partial Explore, title-prefix and expiry indexes
Image compression	Not applicable: text-only product, no image upload or raster assets
Loading skeletons	Route fallback and Explore/paste loading skeletons
Expensive query caching	Short-lived Explore cache; optional shared Redis
N+1 database queries	Avoided: Explore uses one query for one bounded page
Debounced input	Explore search waits 300 ms
Chunk splitting	Lazy routes and Vite vendor chunks
CDN	Fingerprintable static assets and short API cache headers; CDN is provider-configured, not separately proven here
Server-side caching	Redis option with bounded in-memory fallback
Pagination	Bounded page size (maximum 50), stable created_at/id order
API payload compression	Express compression middleware
JS/CSS minification	Vite production build minifies output
Lazy loading	React route pages are dynamically imported
Defer non-critical scripts	No third-party analytics scripts; module entry is deferred by browser
Database connection tooling	Shared pg.Pool, environment-based configuration, Docker Compose local Postgres/Redis

6. Automated tests and build verification

Verification was run against the current repository checkout on 15 October 2026. Server integration tests use PGlite, an in-process PostgreSQL-compatible engine; they do not connect to the user's live Neon database.

Suite	Result
Server tests (node --test)	14 passed, 0 failed
Client tests (Vitest)	3 passed, 0 failed
Production client build (vite build)	Passed; 1,640 modules transformed and optimized chunks emitted
Total automated tests	17 passed, 0 failed

Server cases covered

- Database health success and unavailable responses.
- Required content, input limits, maximum content size, and private-by-default paste creation.
- Short share ID format; full paste retrieval; no-store cache behavior.
- Creation and share-link-guess rate limits.
- Host-header and untrusted-Origin handling.
- Final allowed view behavior, record deletion, and Explore cache invalidation.
- Expiry enforcement; public-only Explore results, pagination, and title-prefix search.
- Explore query caching and cache invalidation after public paste creation.
- Transient database startup retry and final failure propagation.

Client cases covered

Three React user-flow tests passed. The suite covers core paste UI flows; it is not a browser compatibility or end-to-end test against the deployed service.

Verification boundaries

The first sandboxed client-test attempt failed because the runtime could not access the project path while resolving the Vite configuration. The same client suite was rerun with approved local filesystem access and passed. The production build then passed. No live production smoke test, real-Neon connectivity test, load test, or formal security audit is represented as complete by this report.

7. Deployment and operations

The repository includes a Render Blueprint (render.yaml) for a single Node web service. It installs dependencies, builds the React client, starts Express, and uses /api/health for health checks. The server serves the client and API from one origin. Neon supplies managed PostgreSQL through DATABASE_URL; TLS is enabled in Render configuration.

Setting	Purpose
DATABASE_URL	Neon PostgreSQL connection string; store as a Render secret/environment variable.
DATABASE_SSL	Set to true for verified TLS to the managed database.
DB_POOL_MAX	Pool size per application process; default 5. Size against DB limits and instance count.
REDIS_URL	Optional shared Redis connection for cross-instance application cache.
PORT	HTTP listener; supplied by Render.
CLIENT_ORIGIN	Optional single trusted CORS origin; not required for same-origin deployment.

Operational notes

- Local PostgreSQL is not required for the public deployment. Render connects directly to Neon.
- Render free web services may sleep while idle. Neon compute may scale to zero; stored database data remains hosted.
- For multi-instance scaling, use shared Redis and a shared/edge rate-limit store; the current IP limiter is process-local.
- There are no user accounts or ownership controls: anyone with an unlisted paste link can read it. Treat the link as a bearer secret.

8. Repository change history

The commit history shows the application build and database tests, followed by deployment configuration, Render build fixes, endpoint hardening, verified database TLS, pool tuning, retry behavior, and the latest short-link plus final-view cleanup update.

Project links

GitHub repository: github.com/Dinesh-Karthik7736/Pastebin-Lite
Reported public app: pastebox-w5qm.onrender.com

End of report | Prepared from the repository documentation, source files, commit history, and test/build output.